

Estrategia de Testing - ScrapSAE

Extension

Este documento define la estrategia integral de pruebas para asegurar la calidad, estabilidad y correcto funcionamiento de todos los componentes desarrollados en la rama `extension1` (Extensión de Chrome, API, Base de Datos y Landing Page).

La estrategia se divide en tres niveles: **Pruebas Unitarias**, **Pruebas de Integración** y **Pruebas End-to-End (E2E)**.

1. Pruebas Unitarias

Las pruebas unitarias validan el comportamiento de funciones y métodos aislados, sin dependencias externas (sin red, sin base de datos real, sin DOM real si es posible).

1.1 Extensión de Chrome (TypeScript + Vitest)

Se utilizará **Vitest** con **JSDOM** para probar la lógica de la extensión.

Componente	Archivo	Casos de Prueba Principales
Extractor DOM	<code>extractor.ts</code>	- Extracción correcta de texto, imágenes y enlaces dado un HTML mockeado y selectores específicos. - Fallback a selectores secundarios si el primario falla. - Parseo correcto de precios (ej. “\$1,234.56” -> <code>1234.56</code>). - Extracción de variantes desde tablas.
Simulador Humano	<code>human_simulator.ts</code>	- Generación de curvas de Bézier con puntos de control válidos. - Respeto de los límites de tiempo en <code>delay()</code> (mínimo 1 segundo). - Secuencia correcta de eventos en <code>humanClick</code> (mouseenter, mouseover, mousedown, mouseup, click).
Service Worker	<code>service_worker.ts</code>	- Generación correcta del archivo Excel (SheetJS) con el mapeo de columnas configurado. - Aplicación correcta de los límites del plan (ej. bloqueo si se superan las 50 extracciones en plan Free). - Fallback a procesamiento local si la llamada a la IA falla.

1.2 API Backend (C# + xUnit + Moq)

Se utilizará **xUnit** junto con **Moq** para probar los endpoints y servicios del API.

Componente	Archivo	Casos de Prueba Principales
Procesamiento IA	<code>ExtensionEndpoints.cs</code>	- Retorno de <code>BadRequest</code> si la lista de productos está vacía. - Aplicación de <code>ConvertRawToProcessed</code> (fallback) si el servicio de IA lanza una excepción. - Mapeo correcto de campos crudos a procesados.
CRUD Layouts	<code>ExtensionEndpoints.cs</code>	- Asignación automática de <code>Id</code> y <code>CreatedAt</code> en POST. - Serialización correcta a <code>snake_case</code> para Supabase. - Manejo de errores si Supabase retorna un status code no exitoso.
Stripe Webhooks	<code>ExtensionEndpoints.cs</code>	- Actualización correcta del <code>plan_type</code> a “pro” en <code>checkout.session.completed</code>. - Reversión a “free” en <code>customer.subscription.deleted</code>.

2. Pruebas de Integración

Las pruebas de integración validan que los diferentes componentes del sistema se comuniquen correctamente entre sí.

2.1 API + Supabase

Se utilizará **Testcontainers** (o una base de datos de prueba dedicada) para levantar una instancia de PostgreSQL con el esquema de Supabase (`extension_schema.sql`).

Casos de Prueba:

- **Row Level Security (RLS):** Verificar que un usuario no pueda leer ni modificar los layouts de otro usuario.
- **Triggers:** Verificar que al insertar un nuevo usuario en `auth.users` , se cree automáticamente su registro en `public.user_profiles` .

- **Límites de Layouts:** Verificar que la base de datos rechace la inserción de un segundo layout si el usuario está en el plan Free (vía trigger `check_layout_limit`).
- **Layout Default:** Verificar que al marcar un layout como `is_default = true` , los demás layouts del usuario pasen a `is_default = false` .

2.2 Extensión + API

Se utilizará **MSW (Mock Service Worker)** en las pruebas de la extensión para interceptar las llamadas `fetch` y simular las respuestas del API.

Casos de Prueba:

- Envío correcto del payload de productos al endpoint `/api/extension/process` .
 - Manejo adecuado de errores de red (ej. timeout, 500 Internal Server Error) mostrando el mensaje correspondiente en la UI del Popup.
-

3. Pruebas End-to-End (E2E)

Las pruebas E2E validan el flujo completo desde la perspectiva del usuario final, utilizando un navegador real automatizado.

3.1 Landing Page y Checkout (Playwright)

Se utilizará **Playwright** para navegar por la página web.

Casos de Prueba:

- **Navegación:** Verificar que los enlaces del menú funcionen y el scroll suave dirija a las secciones correctas.
- **Responsividad:** Verificar que el menú hamburguesa funcione en resoluciones móviles.
- **Checkout:** Simular el clic en el botón “Upgrade to Pro”, verificar la llamada al API `/api/stripe/create-checkout` y la redirección a la URL de Stripe.

3.2 Extensión de Chrome (Playwright / Puppeteer)

Playwright permite cargar extensiones desempaquetadas en Chromium para probar su UI y comportamiento real.

Casos de Prueba:

- **Flujo de Autenticación:** Abrir el popup, simular login y verificar que se muestre el dashboard con el plan actual.
 - **Gestión de Layouts:** Abrir el SidePanel, crear un nuevo layout, configurar selectores, guardarlo y verificar que aparezca en la lista.
 - **Flujo de Scraping Completo:**
 1. Navegar a una página de prueba estática (ej. un HTML local con productos falsos).
 2. Abrir el popup y hacer clic en “Iniciar Scraping”.
 3. Verificar que el Content Script se inyecte y extraiga los datos.
 4. Verificar que el Service Worker reciba los datos y genere el archivo Excel.
 5. Interceptar la descarga y validar que el archivo `.xlsx` contenga los datos esperados.
-

4. Ejecución y CI/CD

Todas las pruebas automatizadas se integrarán en los flujos de GitHub Actions:

1. **deploy-api.yml** : Ejecutará las pruebas unitarias y de integración de C# (`dotnet test`) antes de compilar y publicar.
 2. **build-extension.yml** : Ejecutará las pruebas unitarias de TypeScript (`pnpm test`) antes de empaquetar los `.zip` .
 3. **Pruebas E2E:** Se configurará un workflow separado (`e2e-tests.yml`) que se ejecutará de forma programada (nightly) o bajo demanda, ya que requieren más tiempo y recursos.
-

5. Plan de Implementación Inmediata

Para validar el desarrollo actual, procederemos a implementar:

1. Configuración de Vitest y pruebas unitarias clave para el Extractor y el Simulador Humano.
2. Configuración de xUnit y pruebas unitarias para los fallbacks del API.
3. Scripts de prueba E2E básicos con Playwright para la Landing Page.